

Go Farm Work - Technical Documentation

1. Architecture Overview

Go Farm Work is a modern, cross-platform application designed to connect farm owners with farm workers in India. The application is built using a modern full-stack web architecture, packaged as both a high-performance web application (PWA/SSR) and a native Android application.

Core Technologies

- **Frontend Framework:** React 19
- **Build Tool:** Vite 8
- **Routing & SSR:** TanStack Router & TanStack Start (Server-Side Rendering)
- **State & Data Fetching:** TanStack Query (React Query)
- **Backend as a Service (BaaS):** Supabase (PostgreSQL)
- **Styling:** Tailwind CSS 4.0
- **UI Components:** Radix UI primitives, Lucide React icons, Framer Motion
- **Forms & Validation:** React Hook Form + Zod
- **Mobile Packaging:** Capacitor v8 (Android)
- **Hosting / Edge Computing:** Cloudflare Pages (via Nitro engine)

2. Infrastructure & Deployment

Database (Supabase)

The application relies on Supabase for data persistence, authentication, and file storage.

- **Auth:** Supabase Auth is used for user authentication (OTP, OAuth).
- **Postgres Database:** Uses a relational data model with Row Level Security (RLS) policies to ensure data privacy.

- **Storage:** Supabase Storage (S3-compatible) is used to host user avatars and other media (`avatars` bucket).
- **Realtime:** Supabase Realtime subscriptions power the live messaging system.

Web Deployment (Cloudflare Pages)

The web version is deployed to Cloudflare Pages utilizing the `nitro` preset. This enables SSR (Server-Side Rendering) at the edge, ensuring extremely fast initial load times and strong SEO performance, even on low-end devices with poor network connections.

Android Application (Capacitor)

The web application is encapsulated into a native Android APK using **Capacitor**.

- Features native splash screens and adaptive icons generated via `@capacitor/assets`.
- Connects to the same Supabase backend.
- Can be built locally using Android Studio or the CLI: `npx cap sync android && cd android && ./gradlew assembleDebug`.

3. Database Schema

The PostgreSQL database is managed via Supabase Migrations (`supabase/migrations/`). Key tables include:

1. **profiles**
 - Extensions to the base `auth.users` table.
 - Fields: `id` (UUID), `full_name`, `phone`, `gender`, `state`, `district`, `taluk`, `village`, `photo_url`, `language`, `onboarded`, `created_at`, `updated_at`.
2. **user_roles**
 - Maps users to their roles.
 - Roles include: `'worker'` and `'landlord'`. A user can have both roles.
3. **jobs**
 - Represents farm work listings.
 - Fields: `id`, `landlord_id`, `title`, `description`, `wage`, `location`, `status` (open, in_progress, completed, cancelled), `created_at`.
4. **job_applications**
 - Connects workers to jobs they have applied for.

- Fields: `id`, `job_id`, `worker_id`, `status` (pending, accepted, rejected), `applied_at`.

5. **messages**

- Peer-to-peer chat system between landlords and workers.
- Fields: `id`, `sender_id`, `receiver_id`, `job_id` (optional), `content`, `read_status`, `created_at`.

6. **reviews**

- Feedback system post-job completion.
- Fields: `id`, `job_id`, `reviewer_id`, `reviewee_id`, `rating` (1-5), `comment`.

Note: All tables are protected by Row Level Security (RLS). Users can only read/write data they own or are permitted to see based on their role and relationships.

4. Key Workflows

Authentication & Onboarding

1. User logs in via Supabase Auth (Google OAuth or Magic Link/OTP).
2. The app checks if `profiles.onboarded` is true.
3. If not, the user is redirected to the `/onboarding` flow to capture:
 - Name, Phone, Gender, Profile Picture.
 - Role Selection (Worker, Landlord, Both).
 - Location (Cascading dropdowns: State > District > Taluk > Village, with manual entry fallback).

Job Lifecycle

1. **Posting:** Landlords post jobs with details (wage, description).
2. **Browsing:** Workers browse jobs filtered by location or wage.
3. **Applying:** Workers apply. The landlord receives a real-time notification.
4. **Acceptance:** Landlord accepts an application, shifting the job status to `in_progress`.
5. **Completion & Review:** Job is marked completed, and escrow/wages are settled. Both parties can leave a 1-5 star review.

5. Environment Variables Configuration

To run this application locally, you must provide the following environment variables in a `.env` file at the root of the project:

```
# Supabase Configuration
VITE_SUPABASE_URL=https://[YOUR_PROJECT_ID].supabase.co
VITE_SUPABASE_ANON_KEY=eyJhbG...

# Optional API Keys (e.g., Google Maps, Analytics)
VITE_GOOGLE_MAPS_API_KEY=...
```

6. Local Development Guide

Prerequisites

- Node.js (v20+)
- npm or pnpm
- Java SDK (for Android build)
- Android Studio & Command Line Tools (for Android build)

Running the Web App

1. Install dependencies: `npm install`
2. Start the development server: `npm run dev`
3. Access at `http://localhost:5173`

Building the Android APK

1. Build the production web bundle: `npm run build`
2. Sync the web assets to the native Android project: `npx cap sync android`
3. Compile the APK:

```
cd android
./gradlew assembleDebug
```

4. The generated APK will be available at: `android/app/build/outputs/apk/debug/app-debug.apk` .